```
#sidebar_b {
position:absolute;
top:82px;
right:10px;
width:25%;
margin-bottom:10px;
padding:10px;
border:1px solid #000;
}
```

For each sidebar, the widths are a little smaller than in previous examples, as we are of course allowing less space for each sidebar. Finally, to prevent sidebar_a from overlapping the footer if there is too much content within it, a left margin identical to that of the content ID needs to be declared for the footer.

```
/* Footer */
#footer {
margin:0 30% 0 30%;
padding:10px;
border:1px solid #000;
background-color:#CCC;
}
```

These result in a perfect positioned three-column layout that stretches to fit the width of the browser window.

9.8 Fixed-Width Layout

Previous layouts in this chapter have been liquid in their approach, with widths declared using percentage values. It is of course perfectly possible and very common, to use set widths declared using pixels. This approach can provide greater control and is still favoured by many web designers. With the troublesome browsers in mind, let's look at an absolute fundamental of CSS layout—the Box Model.

9.9 The Box Model

Before you start using fixed widths for your columns, it is imperative that you get to grips with the Box Model. If you think you might need to apply margins or padding to any of your columns, you need to be aware of the miscalculations these will cause in IE5 and IE5.5 on a PC.

In any standards-compliant browser, the total width for a containing